HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

-----□□🕮□□-----

SOFTWARE ARCHITECTURE AND DESIGN

ASIGNMENT 05 – Academic Microservice Im-plementation

Giảng viên	**: PGS. TS. Trần Đình Quế**
Tên lớp	**: E22CNPM04**
Nhóm bài tập	**: Nhóm 02**
Họ và tên	**: Trương Mạnh Tuân - B22DCVT477**

Hà Nội - 2026

MỤC LỤC

1. Giới thiệu *3*
1.1 Mục tiêu dự án **3**
1.2 Phạm vi hệ thống **3**
2. Kiến trúc hệ thống *4*
2.1 Tổng quan kiến trúc Microservices **4**
2.2 Sơ đồ kiến trúc tổng thể **4**
2.3 Docker Network Topology **5**
3. Công nghệ sử dụng *6*
4. Chi tiết các Microservices *7*
4.1 Danh sách Services **7**
4.2 Giao tiếp giữa các Services (Inter-Service Communication) **8**
5. Thiết kế cơ sở dữ liệu *9*
5.1 Database per Service Pattern **9**
5.2 Data Models **10**
6. API Gateway *13*
6.1 Chức năng **13**
6.2 Bảng định tuyến (Routing Table) **13**
7. API Documentation *14*
7.1 RESTful API Endpoints **14**
7.2 Ví dụ Request/Response **14**
8. Luồng nghiệp vụ chính *16*
8.1 Luồng đặt hàng (Order Creation Flow) **16**
8.2 Luồng đăng ký khách hàng (Customer Registration Flow) **17**
9. Triển khai & Vận hành *17*
9.1 Docker Compose **17**
9.2 Hướng dẫn cài đặt **18**
10. Đánh giá & Hướng phát triển *19*
10.1 Ưu điểm **19**
10.2 Hạn chế & Hướng cải thiện **20**
11. Kết luận *21*
Phụ lục: Cấu trúc thư mục dự án *24*

1. Giới thiệu

1.1 Mục tiêu dự án

Dự án BookStore Microservices được xây dựng với mục tiêu phát triển một hệ thống quản lý nhà sách trực tuyến hoàn chỉnh, áp dụng kiến trúc Microservices hiện đại. Hệ thống cho phép quản lý toàn bộ quy trình kinh doanh của một nhà sách, từ quản lý sản phẩm, khách hàng, đặt hàng, thanh toán đến vận chuyển và đánh giá.

Các mục tiêu cụ thể bao gồm:

- Xây dựng hệ thống theo kiến trúc Microservices với các service độc lập
- Mỗi service có database riêng (Database per Service pattern)
- Triển khai bằng Docker Compose, đảm bảo tính nhất quán môi trường
- Cung cấp API Gateway làm điểm truy cập duy nhất cho client
- Hỗ trợ giao tiếp giữa các service qua REST API nội bộ
- Cung cấp giao diện Web UI để tương tác với hệ thống

1.2 Phạm vi hệ thống

Hệ thống bao gồm 11 microservice độc lập và 1 API Gateway, mỗi service đảm nhận một nghiệp vụ riêng biệt trong quy trình kinh doanh nhà sách:

Bảng 1: Phạm vi chức năng các service

Service	Chức năng
👤 Customer	Quản lý thông tin khách hàng, tự động tạo giỏ hàng
👨‍💼 Staff	Quản lý nhân viên với phân quyền vai trò
👔 Manager	Quản lý cấp quản lý theo phòng ban
📂 Catalog	Phân loại sách theo danh mục/thể loại
📖 Book	Quản lý sách, giá cả và tồn kho
🛒 Cart	Giỏ hàng, quản lý sản phẩm đã chọn
📦 Order	Xử lý đặt hàng, điều phối thanh toán & vận chuyển
💳 Payment	Quản lý thanh toán (credit card, cash, transfer)
🚚 Shipment	Quản lý vận chuyển, theo dõi đơn hàng
💬 Comment	Bình luận và đánh giá sách (1-5 sao)
🤖 Recommender	Gợi ý sách thông minh cho khách hàng

2. Kiến trúc hệ thống

2.1 Tổng quan kiến trúc Microservices

Hệ thống được thiết kế theo kiến trúc Microservices, trong đó mỗi service là một ứng dụng Django độc lập với database riêng, chạy trong Docker container riêng biệt. Tất cả các service giao tiếp với nhau thông qua REST API qua mạng Docker nội bộ (bookstore-net).

Các nguyên tắc thiết kế chính được áp dụng:

Bảng 2: Nguyên tắc thiết kế Microservices

Nguyên tắc	Cách áp dụng
Single Responsibility	Mỗi service chỉ quản lý một domain nghiệp vụ duy nhất
Database per Service	11 database riêng biệt, không chia sẻ schema giữa các service
API Gateway Pattern	Mọi request từ client đều đi qua một entry point duy nhất (port 8000)
Service Orchestration	Order Service đóng vai trò orchestrator, điều phối nhiều service
Loose Coupling	Các service liên kết qua ID (customer_id, book_id), không FK trực tiếp
Independent Deployment	Mỗi service có Dockerfile riêng, build và deploy độc lập

2.2 Sơ đồ kiến trúc tổng thể

Sơ đồ dưới đây mô tả kiến trúc tổng thể của hệ thống BookStore Microservices, bao gồm 3 tầng chính: Client Layer, Microservices Layer và Database Layer.

```
┌──────────────────────────────────────────────────────────┐
│                    🌐 CLIENT LAYER                        │
│          Web Browser (Bootstrap 5)  |  Postman            │
└──────────────────────┬──────────────────────────────────┘
                       │ HTTP :8000
┌──────────────────────▼──────────────────────────────────┐
│              ⚡ API GATEWAY (Django :8000)                 │
│          URL Proxy Routing + Web UI (HTML Templates)      │
└──────┬──────┬──────┬──────┬──────┬──────┬──────────────┘
```

```
   │  │  │  │  │  │  │  │  │  │  │
┌──▼──▼──▼──▼──▼──▼──▼──▼──▼──▼──▼──────────────────────────┐
│              🔧 MICROSERVICES LAYER                        │
│                                                            │
│  Customer  Staff  Manager  Catalog  Book                   │
│   :8001    :8002   :8003    :8004   :8005                   │
│                                                            │
│  Cart    Order   Payment  Shipment                         │
│  :8006   :8007    :8008    :8009                           │
│                                                            │
│  Comment   Recommender                                     │
│   :8010      :8011                                         │
└────────────────────────┬──────────────────────────────────┘
                         │
┌────────────────────────▼──────────────────────────────────┐
│              🗄️ DATABASE LAYER                             │
│            MySQL 8.0 (11 databases)                        │
│  customer_db | staff_db | manager_db | catalog_db          │
│  book_db | cart_db | order_db | pay_db                     │
│  ship_db | comment_db | recommend_db                       │
└────────────────────────────────────────────────────────────┘
```

Hình 1: Sơ đồ kiến trúc tổng thể hệ thống BookStore Microservices

2.3 Docker Network Topology

Toàn bộ 13 container (11 microservices + 1 API Gateway + 1 MySQL) đều nằm trên cùng một Docker network có tên "bookstore-net" (bridge driver). Các service giao tiếp với nhau bằng container name (DNS nội bộ Docker). MySQL sử dụng healthcheck để đảm bảo sẵn sàng trước khi các service khởi động.

```
┌──────────────────── Docker Network: bookstore-net ────────────────────┐
│                                                                        │
│  api-gateway:8000 ──→ customer-service:8001                            │
│                   ──→ staff-service:8002                               │
│                   ──→ manager-service:8003                             │
│                   ──→ catalog-service:8004                             │
│                   ──→ book-service:8005                                │
│                   ──→ cart-service:8006                                │
```

```
│                    ──→ order-service:8007                          │
│                    ──→ pay-service:8008                            │
│                    ──→ ship-service:8009                           │
│                    ──→ comment-rate-service:8010                   │
│                    ──→ recommender-ai-service:8011                 │
│                                                                    │
│  All services ──→ mysql:3306 (healthcheck: mysqladmin ping)        │
└────────────────────────────────────────────────────────────────────┘
```

Hình 2: Docker Network Topology

3. Công nghệ sử dụng

Hệ thống BookStore Microservices sử dụng các công nghệ hiện đại, phổ biến trong phát triển web backend và triển khai containerized applications.

Bảng 3: Công nghệ sử dụng trong hệ thống

Thành phần	Công nghệ	Phiên bản	Mô tả
Ngôn ngữ lập trình	Python	3.11	Ngôn ngữ chính cho tất cả services
Backend Framework	Django	4.2.11	Web framework cho mỗi microservice
REST API	Django REST Framework	3.15.1	Xây dựng RESTful API
Cơ sở dữ liệu	MySQL	8.0	RDBMS cho 11 databases
DB Client	mysqlclient	2.2.4	Python MySQL connector
HTTP Client	requests	2.31.0	Giao tiếp REST giữa các service
CORS	django-cors-headers	4.3.1	Xử lý Cross-Origin requests
Frontend	HTML5 + Bootstrap 5	5.x	Giao diện Web UI responsive
JavaScript	Fetch API (Vanilla)	ES6+	Gọi API từ frontend
Container Runtime	Docker	20.x+	Container hóa các service
Orchestration	Docker Compose	v3.8	Quản lý multi-container

Toàn bộ hệ thống được đóng gói trong Docker containers, sử dụng Docker Compose để orchestrate. Mỗi microservice có Dockerfile riêng dựa trên Python 3.11-slim image, cài đặt các system dependencies (libmysqlclient-dev, gcc) và Python packages qua pip.

```
┌──────────────────────────────────────────────────────────────────┐
│                      🏗️ TECHNOLOGY STACK                          │
│                                                                  │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────────┐  │
│  │  Frontend    │  │ API Gateway │  │    Backend (×11)         │  │
│  ├──────────────┤  ├──────────────┤  ├──────────────────────────┤  │
│  │ HTML5/CSS3 │  │ Django 4.2  │  │ Django 4.2               │  │
│  │ Bootstrap 5│  │ SQLite      │  │ DRF 3.15.1               │  │
│  │ JavaScript │  │ Requests    │  │ MySQL 8.0                │  │
│  │ Fetch API  │  │ URL Proxy   │  │ mysqlclient              │  │
│  └──────────────┘  └──────────────┘  └──────────────────────────┘  │
│                                                                  │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────────┐  │
│  │ Infra        │  │ Language     │  │ Patterns                 │  │
│  ├──────────────┤  ├──────────────┤  ├──────────────────────────┤  │
│  │ Docker       │  │ Python 3.11 │  │ API Gateway Pattern      │  │
│  │ Compose      │  │             │  │ Database per Service     │  │
│  │ MySQL 8.0    │  │             │  │ Service Orchestration    │  │
│  └──────────────┘  └──────────────┘  └──────────────────────────┘  │
└──────────────────────────────────────────────────────────────────┘
```

Hình 3: Technology Stack Overview

4. Chi tiết các Microservices

4.1 Danh sách Services

Hệ thống gồm 12 thành phần (11 microservices + 1 API Gateway), mỗi thành phần chạy trong một Docker container riêng biệt:

Bảng 4: Danh sách các Microservices

#	Service	Port	Database	Mô tả
1	Customer Service	8001	customer_db	Quản lý khách hàng, auto-tạo giỏ hàng
2	Staff Service	8002	staff_db	Quản lý nhân viên nhà sách
3	Manager	8003	manager_db	Quản lý quản lý theo phòng ban

	Service			
4	Catalog Service	8004	catalog_db	Quản lý danh mục/thể loại sách
5	Book Service	8005	book_db	Quản lý sách, giá, tồn kho
6	Cart Service	8006	cart_db	Giỏ hàng + Cart Items
7	Order Service	8007	order_db	Đặt hàng, orchestrate payment & shipment
8	Payment Service	8008	pay_db	Thanh toán đơn hàng
9	Shipment Service	8009	ship_db	Vận chuyển, tracking đơn hàng
10	Comment & Rate	8010	comment_db	Bình luận & đánh giá sách
11	Recommender AI	8011	recommend_db	Gợi ý sách thông minh
12	API Gateway	8000	SQLite	Proxy routing + Web UI

4.2 Giao tiếp giữa các Services (Inter-Service Communication)

Các microservice giao tiếp với nhau qua REST API nội bộ (synchronous HTTP) thông qua Docker network. Dưới đây là danh sách các kết nối inter-service:

Bảng 5: Inter-Service Communication

#	Kết nối	API Call	Mô tả
①	Customer → Cart	POST /carts/	Tự động tạo giỏ hàng rỗng khi đăng ký customer mới
②	Cart → Book	GET /books/{id}/	Kiểm tra sách tồn tại khi thêm vào giỏ
③	Order → Cart	GET /carts/customer/{id}/	Lấy tất cả items trong giỏ hàng
④	Order → Book	GET /books/{id}/	Lấy giá sách để tính tổng đơn hàng
⑤	Order → Payment	POST /payments/	Tự động tạo bản ghi thanh toán (pending)
⑥	Order → Shipment	POST /shipments/	Tự động tạo bản ghi vận chuyển
⑦	Recommender → Book	GET /books/	Lấy danh sách sách để gợi ý ngẫu nhiên

```
Customer ──POST /carts/──────────────────────────────▶ Cart
Cart     ──GET /books/{id}/──────────────────────────▶ Book
Order    ──GET /carts/customer/{id}/──────────────────▶ Cart
Order    ──GET /books/{id}/──────────────────────────▶ Book
Order    ──POST /payments/───────────────────────────▶ Payment
Order    ──POST /shipments/──────────────────────────▶ Shipment
Recommender ──GET /books/────────────────────────────▶ Book
```

Hình 4: Sơ đồ giao tiếp giữa các Services

5. Thiết kế cơ sở dữ liệu

5.1 Database per Service Pattern

Hệ thống áp dụng mô hình "Database per Service" — mỗi microservice sở hữu một database riêng biệt trong cùng MySQL server. Không có service nào truy cập trực tiếp vào database của service khác. Việc liên kết dữ liệu giữa các service được thực hiện thông qua ID tham chiếu (customer_id, book_id, order_id) và REST API calls.

Bảng 6: Database mapping cho các services

Database	Service	Tables
customer_db	Customer Service	customers_customer
staff_db	Staff Service	staffs_staff
manager_db	Manager Service	managers_manager
catalog_db	Catalog Service	catalogs_catalog
book_db	Book Service	books_book
cart_db	Cart Service	carts_cart, carts_cartitem
order_db	Order Service	orders_order, orders_orderitem
pay_db	Payment Service	payments_payment
ship_db	Shipment Service	shipments_shipment
comment_db	Comment Service	comments_comment
recommend_db	Recommender Service	recommendations_recommendation

```
MySQL 8.0 Server (port 3306 internal / 3307 external)
├── customer_db    ← Customer Service :8001
```

```
    ├── staff_db        ← Staff Service :8002
    ├── manager_db      ← Manager Service :8003
    ├── catalog_db      ← Catalog Service :8004
    ├── book_db         ← Book Service :8005
    ├── cart_db         ← Cart Service :8006
    ├── order_db        ← Order Service :8007
    ├── pay_db          ← Payment Service :8008
    ├── ship_db         ← Shipment Service :8009
    ├── comment_db      ← Comment Service :8010
    └── recommend_db    ← Recommender Service :8011
```

Hình 5: Database per Service Architecture

5.2 Data Models

Dưới đây là chi tiết các data model của từng service:

Customer Model:

Field	Type	Ghi chú
id	AutoField	Primary key
name	CharField(100)	Tên khách hàng
email	EmailField	Unique - email khách hàng
phone	CharField(20)	Số điện thoại
address	TextField	Địa chỉ
created_at	DateTimeField	Auto - thời gian tạo

Book Model:

Field	Type	Ghi chú
id	AutoField	Primary key
title	CharField(200)	Tên sách
author	CharField(100)	Tác giả
price	DecimalField(10,2)	Giá sách
stock	IntegerField	Số lượng tồn kho (default: 0)
created_at	DateTimeField	Auto - thời gian tạo

Cart & CartItem Models:

Model	Field	Type	Ghi chú

Cart	id	AutoField	Primary key
Cart	customer_id	IntegerField	ID khách hàng (cross-service ref)
Cart	created_at	DateTimeField	Auto
CartItem	id	AutoField	Primary key
CartItem	cart	ForeignKey(Cart)	FK → Cart
CartItem	book_id	IntegerField	ID sách (cross-service ref)
CartItem	quantity	IntegerField	Số lượng (default: 1)

Order & OrderItem Models:

Model	Field	Type	Ghi chú
Order	id	AutoField	Primary key
Order	customer_id	IntegerField	ID khách hàng
Order	total_amount	DecimalField(10,2)	Tổng tiền đơn hàng
Order	status	CharField	pending/paid/shipped/delivered/cancelled
Order	created_at	DateTimeField	Auto
OrderItem	id	AutoField	Primary key
OrderItem	order	ForeignKey(Order)	FK → Order
OrderItem	book_id	IntegerField	ID sách
OrderItem	quantity	IntegerField	Số lượng
OrderItem	price	DecimalField(10,2)	Giá tại thời điểm mua

Payment Model:

Field	Type	Ghi chú
id	AutoField	Primary key

order_id	IntegerField	ID đơn hàng (cross-service ref)
amount	DecimalField(10,2)	Số tiền thanh toán
method	CharField	credit_card / cash / transfer
status	CharField	pending / completed / failed
created_at	DateTimeField	Auto

Shipment Model:

Field	Type	Ghi chú
id	AutoField	Primary key
order_id	IntegerField	ID đơn hàng
address	TextField	Địa chỉ giao hàng
status	CharField	processing / shipped / delivered
tracking_number	CharField(100)	Mã vận đơn
created_at	DateTimeField	Auto

Comment Model:

Field	Type	Ghi chú
id	AutoField	Primary key
customer_id	IntegerField	ID khách hàng
book_id	IntegerField	ID sách
content	TextField	Nội dung bình luận
rating	IntegerField	Đánh giá 1-5 sao (default: 5)
created_at	DateTimeField	Auto

Recommendation Model:

Field	Type	Ghi chú
id	AutoField	Primary key
customer_id	IntegerField	ID khách hàng
book_id	IntegerField	ID sách được gợi ý
score	FloatField	Điểm gợi ý (0.0 - 1.0)
created_at	DateTimeField	Auto

6. API Gateway

6.1 Chức năng

API Gateway là thành phần trung tâm của hệ thống, đóng vai trò là điểm truy cập duy nhất (single entry point) cho tất cả client requests. Gateway được xây dựng bằng Django và thực hiện các chức năng sau:

- URL Proxy Routing: Chuyển tiếp request từ client đến đúng microservice tương ứng
- Web UI Hosting: Phục vụ các trang HTML (Bootstrap 5) cho quản lý hệ thống
- Service Abstraction: Che giấu topology nội bộ của các microservice khỏi client
- Unified Entry Point: Tất cả API đều truy cập qua cổng 8000

Gateway sử dụng thư viện "requests" của Python để proxy các HTTP request. Khi nhận request tại /api/{service}/{path}, gateway sẽ tra cứu SERVICE_MAP để tìm URL nội bộ tương ứng và chuyển tiếp request.

6.2 Bảng định tuyến (Routing Table)

Bảng 7: API Gateway Routing Table

Client Request	Internal Proxy Target
/api/customers/*	http://customer-service:8001/customers/*
/api/staffs/*	http://staff-service:8002/staffs/*
/api/managers/*	http://manager-service:8003/managers/*
/api/catalogs/*	http://catalog-service:8004/catalogs/*
/api/books/*	http://book-service:8005/books/*
/api/carts/*	http://cart-service:8006/carts/*
/api/cart-items/*	http://cart-service:8006/cart-items/*
/api/orders/*	http://order-service:8007/orders/*
/api/order-items/*	http://order-service:8007/order-items/*
/api/payments/*	http://pay-service:8008/payments/*
/api/shipments/*	http://ship-service:8009/shipments/*
/api/comments/*	http://comment-service:8010/comments/*
/api/recommendations/*	http://recommender-service:8011/recommendations/*
/api/recommend/*	http://recommender-service:8011/recommend/*

7. API Documentation

7.1 RESTful API Endpoints

Tất cả API endpoints tuân theo chuẩn RESTful. Base URL: http://localhost:8000/api. Mỗi service cung cấp các operations CRUD cơ bản (GET, POST, PUT, PATCH, DELETE).

Bảng 8: RESTful API Endpoints

Service	List & Create	Detail (GET/PUT/PATCH/DELETE)
Customer	GET/POST /api/customers/	/api/customers/{id}/
Staff	GET/POST /api/staffs/	/api/staffs/{id}/
Manager	GET/POST /api/managers/	/api/managers/{id}/
Catalog	GET/POST /api/catalogs/	/api/catalogs/{id}/
Book	GET/POST /api/books/	/api/books/{id}/
Cart	GET /api/carts/	/api/carts/{id}/
CartItem	GET/POST /api/cart-items/	/api/cart-items/{id}/
Order	GET/POST /api/orders/	/api/orders/{id}/
OrderItem	GET /api/order-items/	/api/order-items/{id}/
Payment	GET/POST /api/payments/	/api/payments/{id}/
Shipment	GET/POST /api/shipments/	/api/shipments/{id}/
Comment	GET/POST /api/comments/	/api/comments/{id}/
Recommendation	GET/POST /api/recommendations/	/api/recommendations/{id}/

Custom Endpoints (ngoài CRUD cơ bản):

Endpoint	Mô tả
GET /api/carts/customer/{customer_id}/	Lấy giỏ hàng theo Customer ID
GET /api/recommend/	Gợi ý sách ngẫu nhiên (AI Recommendation)

7.2 Ví dụ Request/Response

Tạo khách hàng mới:

```
POST /api/customers/
Content-Type: application/json

{
    "name": "Nguyen Van A",
```

```
    "email": "a@gmail.com",
    "phone": "0901234567",
    "address": "Ha Noi"
}

Response: 201 Created
{
    "id": 1,
    "name": "Nguyen Van A",
    "email": "a@gmail.com",
    "phone": "0901234567",
    "address": "Ha Noi",
    "created_at": "2026-03-09T10:00:00Z"
}
```

Tạo đơn hàng (auto-orchestrate):

```
POST /api/orders/
Content-Type: application/json

{
    "customer_id": 1,
    "address": "123 Nguyen Hue, HCM"
}

Response: 201 Created
{
    "id": 1,
    "customer_id": 1,
    "total_amount": "44.48",
    "status": "pending",
    "items": [
        {"book_id": 1, "quantity": 2, "price": "15.99"},
        {"book_id": 2, "quantity": 1, "price": "12.50"}
    ]
}
→ Auto-created: Payment (pending) + Shipment (processing)
```

8. Luồng nghiệp vụ chính

8.1 Luồng đặt hàng (Order Creation Flow)

Đây là luồng nghiệp vụ phức tạp nhất trong hệ thống, thể hiện rõ mô hình Service Orchestration. Khi client gửi request tạo đơn hàng, Order Service đóng vai trò orchestrator và điều phối 4 service khác:

```
Client ──POST /api/orders/──▶ Gateway ──▶ Order Service
                                                  │
                        ┌─────────────────────────┘
                        │
              Step 1: │──GET /carts/customer/{id}/──▶ Cart Service
                        │   ◀── {items: [{book_id, qty}]}
                        │
              Step 2: │──GET /books/{book_id}/──▶ Book Service (loop)
                        │   ◀── {price, stock}
                        │
              Step 3: │  Calculate: total = Σ(price × qty)
                        │  Save Order + OrderItems to order_db
                        │
              Step 4: │──POST /payments/──▶ Payment Service
                        │   ◀── {id, status: "pending"}
                        │
              Step 5: │──POST /shipments/──▶ Shipment Service
                        │   ◀── {id, status: "processing"}
                        │
                        └──▶ Response: 201 {order + items}
```

Hình 6: Sequence Diagram — Luồng đặt hàng

Chi tiết các bước:

- Bước 1: Order Service gọi Cart Service để lấy danh sách items trong giỏ hàng của customer
- Bước 2: Với mỗi item, Order Service gọi Book Service để lấy giá sách hiện tại
- Bước 3: Tính tổng tiền (total_amount = Σ price × quantity), lưu Order và OrderItems
- Bước 4: Tự động tạo bản ghi Payment (status: pending, method: credit_card)
- Bước 5: Tự động tạo bản ghi Shipment (status: processing, address từ request)

8.2 Luồng đăng ký khách hàng (Customer Registration Flow)

Khi một khách hàng mới được tạo, Customer Service tự động gọi Cart Service để tạo một giỏ hàng rỗng cho khách hàng đó:

```
Client ──POST /api/customers/──▶ Gateway ──▶ Customer Service
                                                    │
                                Save customer ◀─────┘
                                to customer_db
                                      │
                                      │──POST /carts/──▶ Cart Service
                                      │   {customer_id: new_id}
                                      │   ◀── {id, customer_id}
                                      │
                                      └──▶ Response: 201 {customer data}
```

Hình 7: Sequence Diagram — Luồng đăng ký khách hàng

Tóm tắt toàn bộ Business Flow:

- 1. Tạo Customer → Auto-tạo Cart rỗng
- 2. Tạo Books → Thêm sách vào hệ thống
- 3. Add CartItem → Thêm sách vào giỏ (verify book tồn tại)
- 4. Create Order → Fetch cart → Fetch prices → Create order → Auto Payment → Auto Shipment
- 5. Add Comment → Đánh giá sách (1-5 sao)
- 6. Get Recommendations → Gợi ý sách cho khách hàng

9. Triển khai & Vận hành

9.1 Docker Compose

Toàn bộ hệ thống được triển khai bằng Docker Compose (version 3.8). File docker-compose.yml định nghĩa 13 services, 1 network và 1 volume:

Bảng 9: Docker Compose Services

Container	Image / Build	Port Mapping	Dependency
mysql	mysql:8.0	3307:3306	Database server với healthcheck
customer-	Build ./customer-	8001:8001	depends_on: mysql (healthy)

service	service		
staff-service	Build ./staff-service	8002:8002	depends_on: mysql (healthy)
manager-service	Build ./manager-service	8003:8003	depends_on: mysql (healthy)
catalog-service	Build ./catalog-service	8004:8004	depends_on: mysql (healthy)
book-service	Build ./book-service	8005:8005	depends_on: mysql (healthy)
cart-service	Build ./cart-service	8006:8006	depends_on: mysql (healthy)
order-service	Build ./order-service	8007:8007	depends_on: mysql (healthy)
pay-service	Build ./pay-service	8008:8008	depends_on: mysql (healthy)
ship-service	Build ./ship-service	8009:8009	depends_on: mysql (healthy)
comment-rate-service	Build ./comment-rate-service	8010:8010	depends_on: mysql (healthy)
recommender-ai-service	Build ./recommender-ai-service	8011:8011	depends_on: mysql (healthy)
api-gateway	Build ./api-gateway	8000:8000	depends_on: all 11 services

MySQL sử dụng healthcheck (mysqladmin ping) với interval 10s, timeout 5s, retries 10, start_period 30s. Tất cả microservices chỉ khởi động khi MySQL healthy. Các service có restart: on-failure để tự khởi động lại nếu gặp lỗi.

9.2 Hướng dẫn cài đặt

Yêu cầu hệ thống:

- Docker ≥ 20.x và Docker Compose ≥ 2.x
- RAM ≥ 4GB (13 containers + MySQL)
- Cổng 8000-8011 và 3307 khả dụng

Các bước cài đặt:

```
# Bước 1: Di chuyển vào thư mục
cd bookstore-micro05
```

```
# Bước 2: Build và khởi chạy
docker-compose up --build -d

# Bước 3: Kiểm tra trạng thái
docker-compose ps

# Bước 4: Chạy migrations (lần đầu)
for svc in customer-service staff-service manager-service \
  catalog-service book-service cart-service order-service \
  pay-service ship-service comment-rate-service \
  recommender-ai-service; do
  docker-compose exec $svc python manage.py makemigrations
  docker-compose exec $svc python manage.py migrate
done

# Bước 5: Truy cập
# Web UI:  http://localhost:8000
# API:     http://localhost:8000/api/books/

# Dừng hệ thống
docker-compose down        # Giữ data
docker-compose down -v     # Xóa data
```

10. Đánh giá & Hướng phát triển

10.1 Ưu điểm

Bảng 10: Ưu điểm của hệ thống

Ưu điểm	Mô tả
Kiến trúc Microservices	Mỗi service độc lập, có thể phát triển, test và deploy riêng biệt. Team khác nhau có thể làm việc song song trên các service khác nhau.
Database per Service	Dữ liệu được tách biệt hoàn toàn giữa các service, đảm bảo tính encapsulation và autonomy. Mỗi service có thể chọn database phù hợp.
API Gateway	Client chỉ cần biết một endpoint duy nhất (port 8000), không cần biết topology nội bộ. Dễ dàng thêm authentication, rate limiting.

Docker Compose	Toàn bộ hệ thống khởi động bằng 1 lệnh. Đảm bảo consistency giữa development, testing và production environments.
Service Orchestration	Order Service thể hiện rõ pattern orchestration, điều phối nhiều service để hoàn thành nghiệp vụ phức tạp.
RESTful API	Tất cả service tuân theo chuẩn REST, dễ tích hợp với bất kỳ client nào (web, mobile, third-party).
Web UI	Cung cấp giao diện Bootstrap 5 responsive, cho phép thao tác CRUD trực quan mà không cần công cụ bên ngoài.

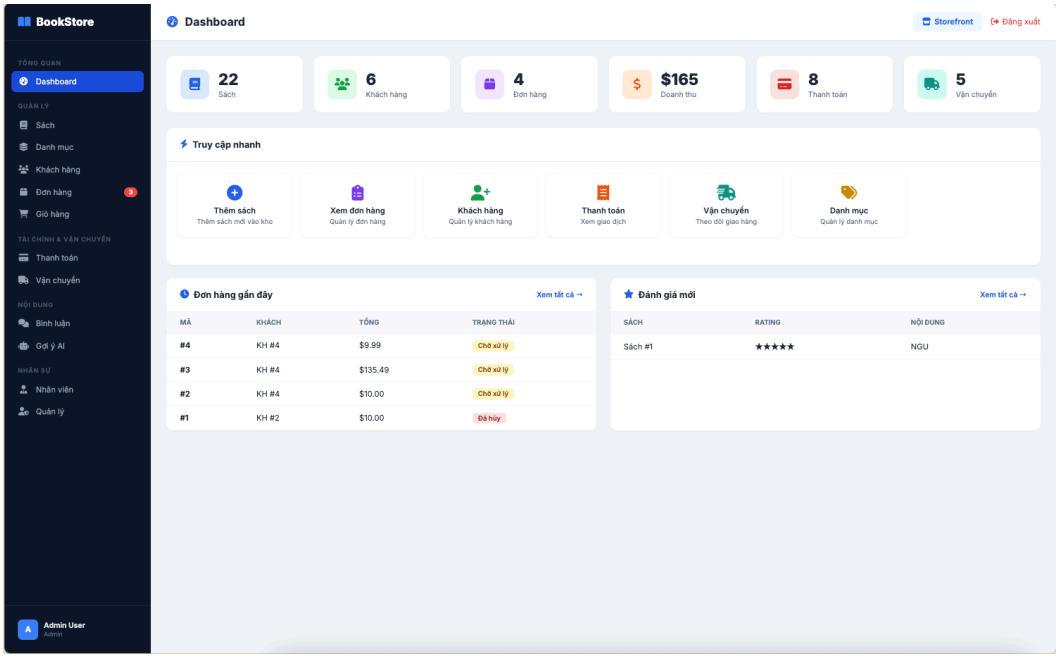
10.2 Hạn chế & Hướng cải thiện

Bảng 11: Hạn chế và hướng cải thiện

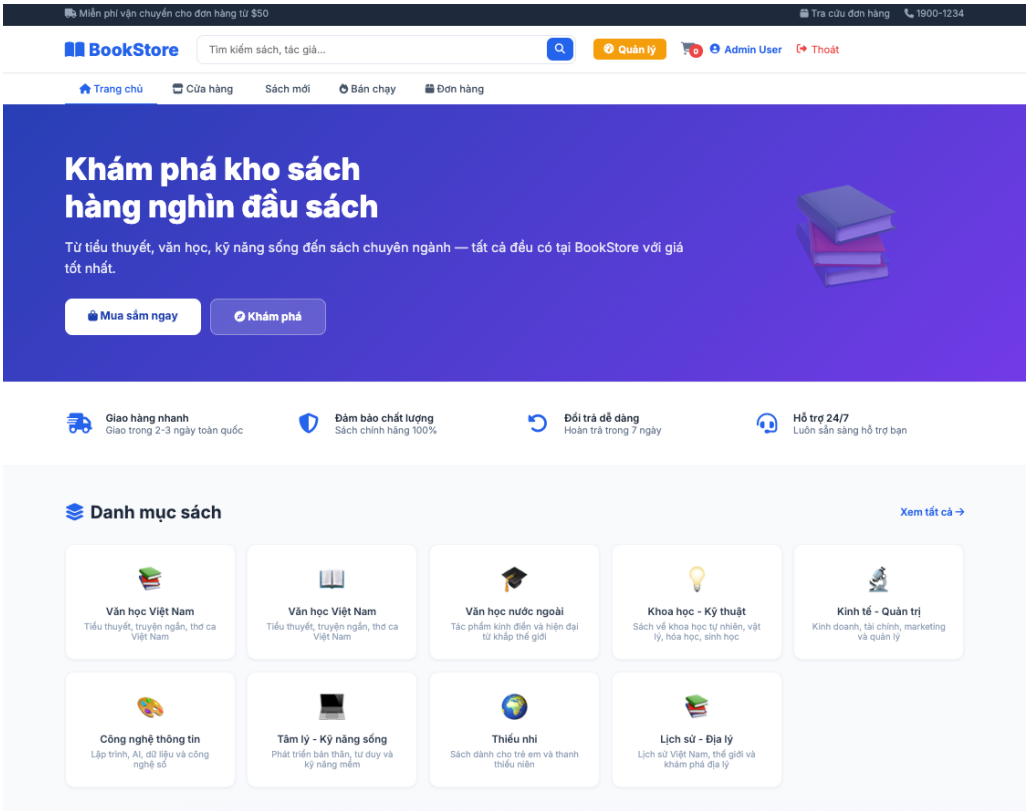
Hạn chế	Chi tiết	Hướng cải thiện
Synchronous Communication	Tất cả inter-service calls là synchronous REST, có thể gây cascading failures	Thêm Message Queue (RabbitMQ/Kafka) cho async
No Authentication	API không có authentication/authorization	Thêm JWT/OAuth2 tại API Gateway
No Service Discovery	Service URLs hardcoded trong code	Thêm Consul/Eureka cho dynamic discovery
No Circuit Breaker	Không có cơ chế fallback khi service down	Thêm Circuit Breaker pattern (retry + fallback)
No Centralized Logging	Logs phân tán trong các container	Thêm ELK Stack hoặc Loki cho centralized logging
No Monitoring	Không có metrics/alerting	Thêm Prometheus + Grafana cho monitoring
Simple Recommender	Gợi ý sách chỉ là random	Thêm ML-based recommendation engine
No Data Consistency	Không có saga pattern cho distributed transactions	Implement Saga pattern hoặc Event Sourcing

11. Demo

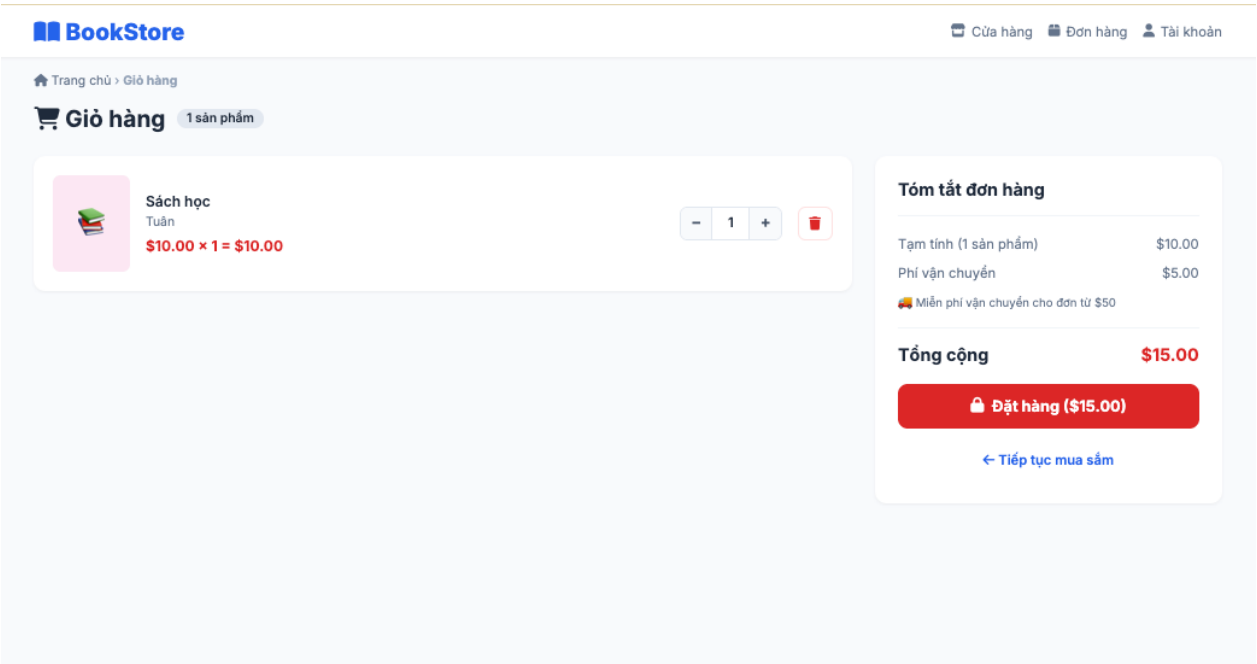
Dashboard



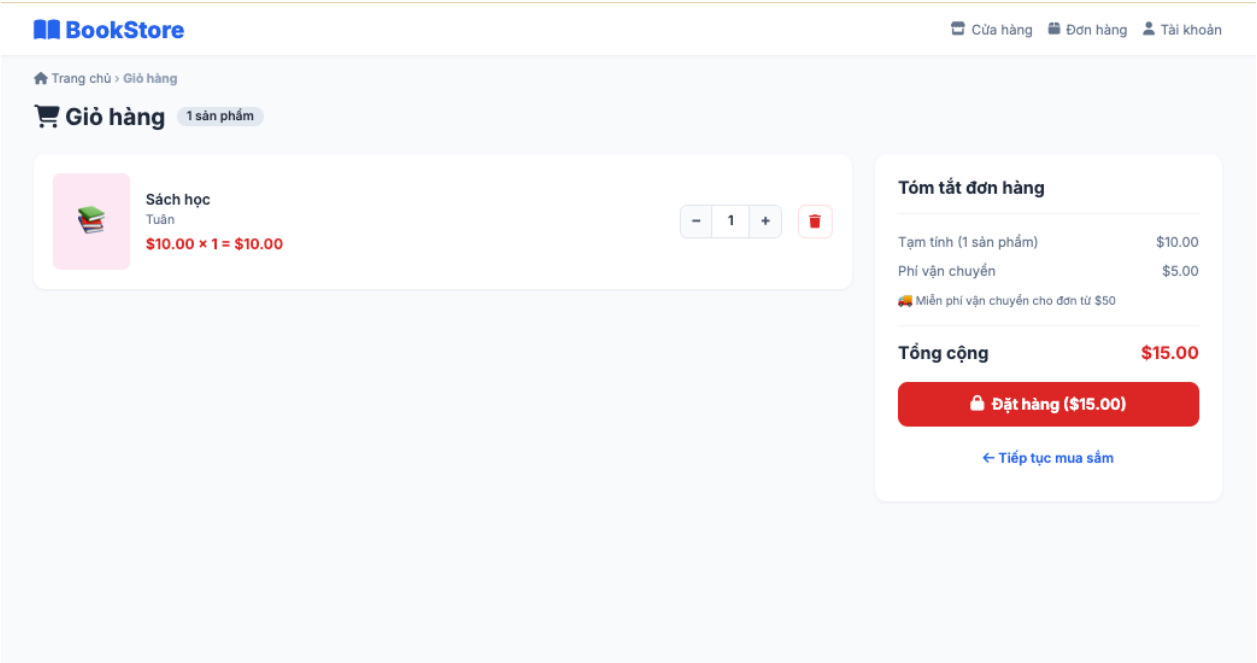
Trang chủ



Giỏ hàng



Sách



12. Kết luận

Hệ thống BookStore Microservices đã được xây dựng thành công với kiến trúc microservices hoàn chỉnh, bao gồm 11 dịch vụ độc lập và 1 API Gateway. Hệ thống thể hiện rõ các nguyên tắc thiết kế microservices quan trọng:

- Single Responsibility: Mỗi service đảm nhận đúng một nghiệp vụ
- Database per Service: 11 database riêng biệt, đảm bảo data isolation
- API Gateway Pattern: Một entry point duy nhất cho client
- Service Orchestration: Order Service điều phối quy trình đặt hàng phức tạp
- Containerization: Toàn bộ hệ thống đóng gói trong Docker containers

Hệ thống đã triển khai thành công trên Docker Compose với 13 containers hoạt động ổn định, bao gồm MySQL 8.0 với healthcheck, 11 Django microservices và 1 API Gateway với Web UI. Các tính năng chính hoạt động đầy đủ: quản lý khách hàng, sách, giỏ hàng, đặt hàng tự động (orchestration), thanh toán, vận chuyển, đánh giá và gợi ý sách.

Dự án cũng đi kèm với bộ tài liệu đầy đủ bao gồm: API Documentation chi tiết cho tất cả endpoints, Architecture Diagrams (Mermaid) cho sơ đồ kiến trúc, và Postman Collection với 50+ requests sẵn sàng test. Đây là nền tảng tốt để tiếp tục phát triển thêm các tính năng nâng cao như authentication, message queue, monitoring và machine learning-based recommendations trong tương lai.

— Hết —

Phụ lục: Cấu trúc thư mục dự án

```
bookstore-micro05/
├── docker-compose.yml
├── init.sql
├── api-gateway/
│   ├── Dockerfile
│   ├── manage.py
│   ├── requirements.txt
│   ├── gateway_project/
│   │   ├── settings.py
│   │   └── urls.py
│   ├── gateway/
│   │   ├── views.py
│   │   └── urls.py
│   └── templates/
│       ├── index.html
│       ├── books.html
│       ├── cart.html
│       ├── customers.html
│       ├── orders.html
│       ├── payments.html
│       ├── shipments.html
│       ├── staffs.html
│       ├── managers.html
│       ├── catalogs.html
│       ├── comments.html
│       └── recommender.html
├── customer-service/
│   ├── Dockerfile
│   ├── manage.py
│   ├── requirements.txt
│   ├── customer_project/settings.py
│   └── customers/
│       ├── models.py
│       ├── serializers.py
│       ├── views.py
│       ├── urls.py
│       └── migrations/
```

```
├── staff-service/          (same structure)
├── manager-service/        (same structure)
├── catalog-service/        (same structure)
├── book-service/           (same structure)
├── cart-service/           (same structure)
├── order-service/          (same structure)
├── pay-service/            (same structure)
├── ship-service/           (same structure)
├── comment-rate-service/   (same structure)
├── recommender-ai-service/ (same structure)
├── API_DOCUMENTATION.md
├── ARCHITECTURE_DIAGRAM.md
├── BookStore_API.postman_collection.json
└── README.md
```